

Full Stack Developer (MERN Stack)

Project Title: ShopSmart- Your Digital Grocery Store

Experience

Team ID: LTVIP2025TMID48196

Team Size: 4

1.Team Leader: BHUMANAPALLI VENKATA TEJ LOHITH REDDY
(Frontend Developer)

Role: Builds the visual and interactive parts of a website or application that users interact with directly. They use technologies like HTML, CSS, JavaScript, and frameworks such as React or Angular to create responsive, user-friendly interfaces.

2. Team Member: BUMMANNA GARI YELLA PRASHANTHI(Backend Developer)

Role: Builds and maintains the server-side logic, databases, and APIs that power a web application.They work with languages like Node.js, Python, Java and tools like MongoDB.

3. Team Member:B SRAVANTHI (Full Stack Developer)

Role: Handles both frontend and backend development, building complete web applications end-to-end. They work with technologies like HTML, CSS, JavaScript for the frontend and Node.js, Python, or Java with databases on the backend.

4. Team Member:BANDA NESE PRIYANKA (Database Administration)

Role: Involves managing and maintaining databases to ensure their performance, security, and reliability. A Database Administrator uses tools like MongoDB to design, implement, backup, and recover data systems.

Introduction:

With the rise of e-commerce and the digital economy, grocery shopping has evolved into a more convenient and accessible online experience. This project, *ShopSmart*, is a full-stack grocery web application that allows users to browse, search, and purchase groceries from the comfort of their homes, while enabling admins to manage products, categories, and orders.

Project Description:

our basic grocery-web app! Our app is designed to provide a seamless online shopping experience for customers, making it convenient for them to explore and purchase a wide range of products. Whether you are a tech enthusiast, a fashionista, or a homemaker looking for everyday essentials, our app has something for everyone. With user-friendly navigation and intuitive design, our grocery-webapp app allows customers to browse through various categories, view product details, add items to their cart, and securely complete the checkout process. We prioritize user satisfaction and aim to provide a smooth and hassle-free shopping experience.

For sellers and administrators, our app offers robust backend functionalities. Sellers can easily manage their product listings, inventory, and orders, while administrators can efficiently handle customer inquiries, process payments, and monitor overall app performance.

With a focus on security and privacy, our grocery-webapp app ensures that customer data is protected, transactions are secure, and personal information remains confidential. We strive to build trust with our customers and provide a safe platform for online shopping. We are excited to have you on board and look forward to providing you with a delightful shopping experience. Happy shopping with our grocery-webapp!

Scenario Based Case Study:

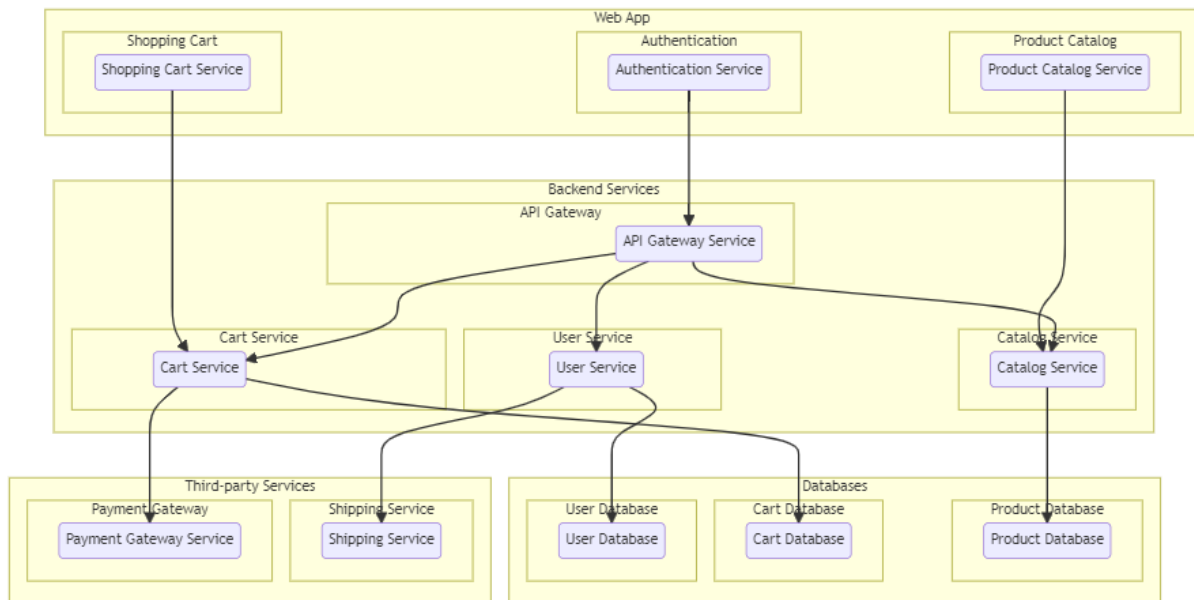
Customer Experience:

- **Profile:** Ravi, a 32-year-old IT professional, prefers online grocery shopping due to his busy schedule.
- **Action:** He registers, logs in, browses categories, searches for products like "organic tomatoes", adds items to his cart, and checks out using Cash on Delivery.
- **Experience:**
 - Easy product filtering and responsive UI
 - Quick checkout and order confirmation
 - Real-time order tracking
- **Outcome:** Ravi receives his order on time and rates the app 5 stars for convenience and user-friendliness.

Admin Experience:

- **Profile:** Anitha, 28-year old manages the store backend and handles product and order management.
- **Action:** She logs into the admin dashboard, adds new products, checks Ravi's order, updates its status (Packed → Out for Delivery → Delivered), and monitors low-stock items.
- **Experience:**
 - Simple dashboard interface
 - Easy stock and order management
 - Weekly reports for analytics
- **Outcome:** Efficient order processing, timely delivery, and improved business insights.

Technical Architecture:-



Frontend: Built using Angular/React for dynamic UI rendering.

Backend: Express.js handles routing, APIs, authentication, and order processing.

Database: MongoDB stores product data, user info, and orders.

Authentication: JWT-based login for users and admins.

The technical architecture of a flower and gift delivery app typically involves a client-server model, where the frontend represents the client and the backend serves as the server. The frontend is responsible for user interface, interaction, and presentation, while the backend handles data storage, business logic, and integration with external services like payment gateways and databases. Communication between the frontend and backend is typically facilitated through APIs, enabling seamless data exchange and functionality.

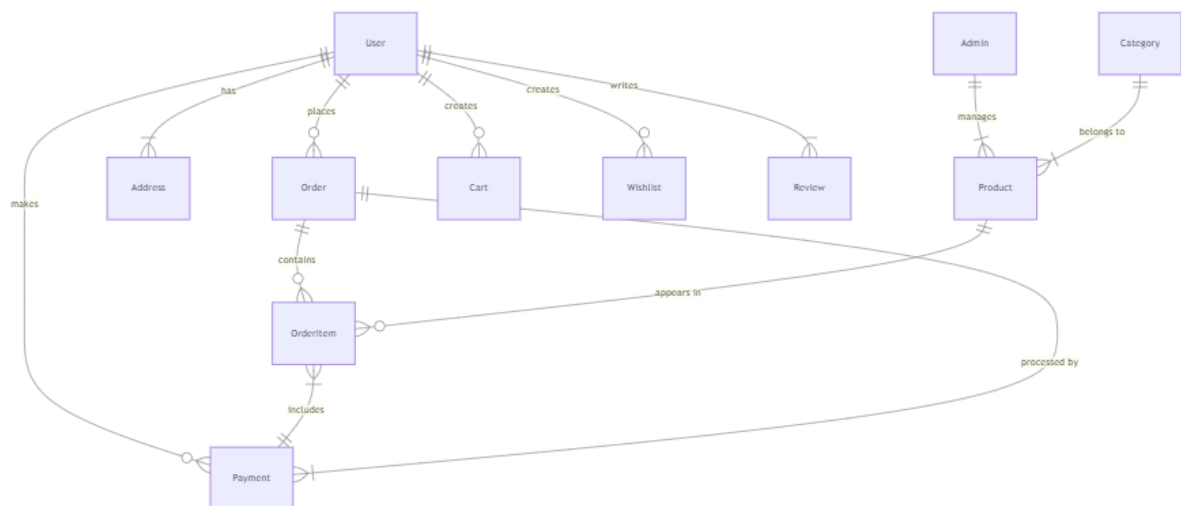
ER-Diagram:

Entities:

- **User** (userID, name, email, password, address)
- **Product** (productID, name, category, price, stock, image)
- **Order** (orderID, userID, productIDs[], totalAmount, orderDate, status)
- **Admin** (adminID, name, email, password)

Relationships:

- A **User** can place many **Orders**.
- An **Order** contains many **Products**.
- **Admins** can manage **Products** and **Orders**.



The Entity-Relationship (ER) diagram for an flower and gift delivery app visually represents the relationships between different entities involved in the system, such as users, products, orders, and reviews. It illustrates how

these entities are related to each other and helps in understanding the overall database structure and data flow within the application.

Key Features:

Product Catalog:

Our grocery web app offers a wide-ranging product catalog organized into multiple categories and subcategories. Users can easily search, browse, and filter products based on preferences like category, brand, or rating, making product discovery seamless and efficient.

Shopping Cart & Checkout:

The app features a robust shopping cart where users can:

- Add or remove products
- Adjust quantities
- View total pricing

During checkout, users are guided through a secure and user-friendly payment process, with support for multiple payment methods such as UPI, card, and COD.

Product Reviews & Ratings:

Users can **rate** products and write **reviews** based on their experience. This community-driven feature helps other customers make **informed decisions**, boosts trust, and encourages interaction.

Order Tracking:

After placing an order, users receive **real-time updates** on their order status including:

- Order confirmed
- Packed
- Shipped
- Out for delivery
- Delivered

This ensures full **transparency** and keeps customers informed every step of the way.

Admin Dashboard:

Admins are provided with a **comprehensive dashboard** to:

- Manage products and categories
- Update inventory and pricing
- Handle customer queries
- Monitor order statuses

The dashboard also includes **sales analytics** and **customer insights** to support informed business decisions.

Order Management:

The system manages the full **order lifecycle**, including:

- Order placement
- Status updates
- Shipment tracking

Users can also **view order history**, and request **returns or cancellations**, providing them full control over their orders.

Advanced Search & Filters:

The search functionality allows users to find products using **keywords** and then narrow results using **filters** such as:

- Price range
- Brand
- Product rating

This ensures fast and accurate access to what they're looking for.

PROJECT STRUCTURE:

This structure assumes an Angular app and follows a modular approach. Here's a brief explanation of the main directories and files:

- `src/app/components`: Contains components related to the customer app, such as register, login, home, products, my-cart, my-orders, placeorder, history, feedback, product-details, and more.
- `src/app/modules`: Contains modules for different sections of the app. In this case, the admin module is included with its own set of components like add-category, add-product, dashboard, feedback, home, orders, payment, update-product, users, and more.
- `src/app/app-routing.module.ts`: Defines the routing configuration for the app, specifying which components should be loaded for each route.
- `src/app/app.component.ts`, `src/app/app.component.html`, `src`.

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>ShopSmart - Digital Grocery Store</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f4f4f4; padding: 20px; }
    h1 { text-align: center; color: #2c3e50; }
    table { width: 100%; border-collapse: collapse; background: white; margin-top: 20px; }
    th, td { padding: 12px; text-align: center; border: 1px solid #ccc; }
    img { width: 80px; height: auto; border-radius: 8px; }
    button { margin: 2px; padding: 8px 12px; border: none; border-radius: 5px; cursor: pointer; }
    .add-btn { background-color: #28a745; color: white; }
    .buy-btn { background-color: #007bff; color: white; }
    .delete-btn { background-color: #dc3545; color: white; }
    #cart { margin-top: 30px; background: #fff; padding: 15px; border: 1px solid #ccc; }
    #cartItems li { margin: 5px 0; }
  </style>
</head>
```


Pre-requisites:

To develop a full-stack Grocery web app using AngularJS, Node.js, and MongoDB, there are several prerequisites you should consider. Here are the key prerequisites for developing such an application:

Node.js and npm: Install Node.js, which includes npm (Node Package Manager), on your development machine. Node.js is required to run JavaScript on the server side.

- Download: <https://nodejs.org/en/download/>
- Installation instructions: <https://nodejs.org/en/download/package-manager/>

MongoDB: Set up a MongoDB database to store hotel and booking information. Install MongoDB locally or use a cloud-based MongoDB service.

- Download: <https://www.mongodb.com/try/download/community>
- Installation instructions: <https://docs.mongodb.com/manual/installation/>

Express.js: Express.js is a web application framework for Node.js. Install Express.js to handle server-side routing, middleware, and API development.

- Installation: Open your command prompt or terminal and run the following command: `npm install express`

Angular: Angular is a JavaScript framework for building client-side applications. Install Angular CLI (Command Line Interface) globally to create and manage your Angular project.

Install Angular CLI:

- Angular provides a command-line interface (CLI) tool that helps with project setup and development.
- Install the Angular CLI globally by running the following command:

```
npm install -g @angular/cli
```

Verify the Angular CLI installation:

- Run the following command to verify that the Angular CLI is installed correctly: `ng version`.
- You should see the version of the Angular CLI printed in the terminal if the installation was successful.

Create a new Angular project:

- Choose or create a directory where you want to set up your Angular project.
- Open your terminal or command prompt.
- Navigate to the selected directory using the `cd` command.
- Create a new Angular project by running the following command: `ng new client` Wait for the project to be created:
- The Angular CLI will generate the basic project structure and install the necessary dependencies

Navigate into the project directory:

- After the project creation is complete, navigate into the project directory by running the following command: `cd client`

Start the development server:

- To launch the development server and see your Angular app in the browser, run the following command: `ng serve / npm start`
- The Angular CLI will compile your app and start the development server.
- Open your web browser and navigate to `http://localhost:4200` to see your Angular app running.

You have successfully set up Angular on your machine and created a new Angular project. You can now start building your app by modifying the generated project files in the `src` directory.

Please note that these instructions provide a basic setup for Angular. You can explore more advanced configurations and features by referring to the official Angular documentation: <https://angular.io>

HTML, CSS, and JavaScript: Basic knowledge of HTML for creating the structure of your app, CSS for styling, and JavaScript for client-side interactivity is essential.

Database Connectivity: Use a MongoDB driver or an Object-Document Mapping (ODM) library like Mongoose to connect your Node.js server with the MongoDB database and perform CRUD (Create, Read, Update, Delete) operations.

Front-end Framework: Utilize Angular to build the user-facing part of the application, including products listings, booking forms, and user interfaces for the admin dashboard.

Version Control: Use Git for version control, enabling collaboration and tracking changes throughout the development process. Platforms like GitHub or Bitbucket can host your repository.

Development Environment: Choose a code editor or Integrated Development Environment (IDE) that suits your preferences, such as Visual Studio Code, Sublime Text, or WebStorm.

- **Visual Studio Code:** Download from <https://code.visualstudio.com/download>
- **Sublime Text:** Download from <https://www.sublimetext.com/download>
- **WebStorm:** Download from <https://www.jetbrains.com/webstorm/download>

To Connect the Database with Node JS go through the below provided link:

- Link: <https://www.section.io/engineering-education/nodejs-mongoosejs-mongodb/>

Git Repository Cloning:

To run the existing grocery-web app project downloaded from github:

Follow below steps:

1. Clone the Repository:

- Open your terminal or command prompt.
- Navigate to the directory where you want to store the grocery-webapp app.
- Execute the following command to clone the repository:

git clone: [_](#)

2. Install Dependencies:

- Navigate into the cloned repository directory:

```
cd grocery-webapp
```

- Install the required dependencies by running the following command:

```
npm install
```

3. Start the Development Server:

- To start the development server, execute the following command:

```
npm run dev or npm run start
```

- The e-commerce app will be accessible at <http://localhost:5100> by default. You can change the port configuration in the `.env` file if needed.

4. Access the App:

- Open your web browser and navigate to <http://localhost:5100>.
- You should see the grocery-webapp app's homepage, indicating that the installation and setup were successful.

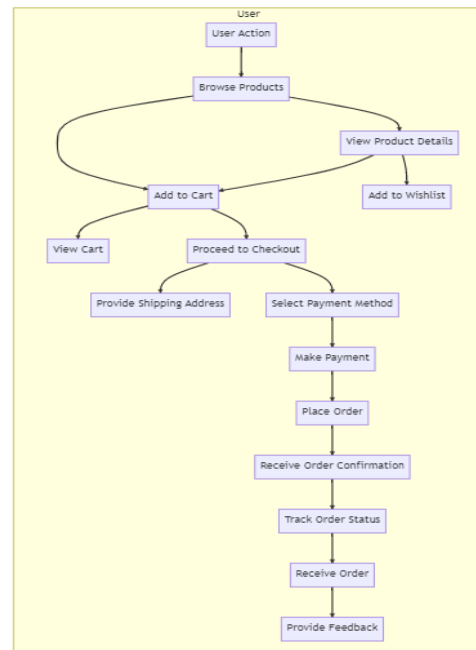
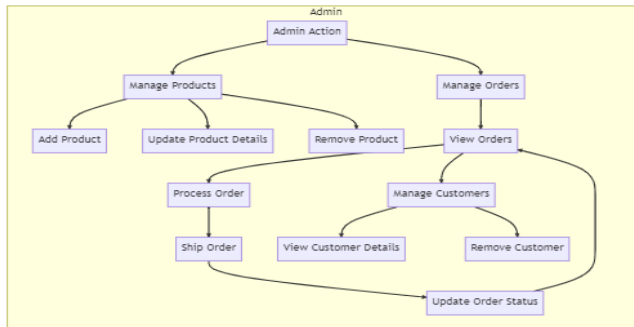
Video Tutorial Link to clone the project:

Project Repository Link:

Congratulations! You have successfully installed and set up the grocery-webapp app on your local machine. You can now proceed with further customization, development, and testing as needed.

Role Based Access:

Roles of Admin and User can be defined for an online grocery web application



Roles and Responsibility:

User:

- Register and log in
- Browse products
- Add/remove from cart
- Place orders
- Track and review orders

Admin:

- Manage products and categories
- Handle stock levels
- View user list and orders
- Update order status (Packed, Shipped, Delivered)
- Generate reports

Project Flow

- Frontend Development
- Backend Development
- Integration

Frontend Development:

Frontend development involves building the user interface (UI) and implementing the visual elements of the online shopping web application. It focuses on creating an intuitive and engaging user experience that allows users to interact with the application seamlessly.

To Setup the frontend development and to connect node.js with MongoDB Database

User Interface (UI) Design

- Create a visually appealing and consistent design using modern design principles.
- Use a UI design tool like Adobe XD, Sketch, Figma, or InVision to create wireframes and mockups.
- Pay attention to typography, color schemes, spacing, and visual hierarchy.
- Use responsive design techniques to ensure the app looks great on different devices.

Responsive Design

- Utilize CSS media queries and responsive design frameworks like Bootstrap or Tailwind CSS to create a responsive layout.
- Test your app on various devices and screen sizes to ensure a seamless user experience.

Product Catalog

- Design and implement a product listing page that displays product images, titles, descriptions, prices, and other relevant details.
- Implement search functionality to allow users to find products easily.

- Include filters and sorting options to enhance the browsing experience.

Shopping Cart and Checkout Process

- Design and develop a shopping cart component to allow users to add products, view cart contents, update quantities, and remove items.
- Create a checkout process with multiple steps, including shipping information, payment selection, and order review.

User Authentication and Account Management

- Design and implement a user registration and login system.
- Create user profile pages where users can view and edit their personal information, addresses, payment methods, and order history.
- Implement authentication guards to restrict access to certain pages or features.

Payment Integration

- Integrate with a payment gateway service like Stripe, PayPal, or Braintree.
- Implement a secure and seamless payment flow that allows users to enter payment details and complete transactions.
- Handle transaction success and failure scenarios and provide appropriate feedback to the user.

Backend Development:

Backend development involves building the server-side components and logic of the online shopping web application. It focuses on handling the business logic, processing requests from the front end, and interacting with the database. The following activities are part of the backend development

process:

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>ShopSmart - Digital Grocery Store</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f4f4f4; padding: 20px; }
    h1 { text-align: center; color: #2c3e50; }
    table { width: 100%; border-collapse: collapse; background: white; margin-top: 20px; }
    th, td { padding: 12px; text-align: center; border: 1px solid #ccc; }
    img { width: 80px; height: auto; border-radius: 8px; }
    button { margin: 2px; padding: 8px 12px; border: none; border-radius: 5px; cursor: pointer; }
    .add-btn { background-color: #28a745; color: white; }
    .buy-btn { background-color: #007bff; color: white; }
    .delete-btn { background-color: #dc3545; color: white; }
    #cart { margin-top: 30px; background: #fff; padding: 15px; border: 1px solid #ccc; }
    #cartItems li { margin: 5px 0; }
  </style>
</head>
```

Set Up Backend:

Set Up Project Structure:

- Create a new directory for your project and set up a package.json file using npm init command.
- Install necessary dependencies such as Express.js, Mongoose, and other required packages.

Database Configuration:

- Set up a MongoDB database either locally or using a cloud-based MongoDB service like MongoDB Atlas.
- Create a database and define the necessary collections for hotels, users, bookings, and other relevant data.

Create Express.js Server:

- Set up an Express.js server to handle HTTP requests and serve API endpoints.
- Configure middleware such as body-parser for parsing request bodies and cors for handling cross-origin requests.

Define API Routes:

- Create separate route files for different API functionalities such as hotels, users, bookings, and authentication.

- Define the necessary routes for listing hotels, handling user registration and login, managing bookings, etc.
- Implement route handlers using Express.js to handle requests and interact with the database.

Implement Data Models:

- Define Mongoose schemas for the different data entities like hotels, users, and bookings.
- Create corresponding Mongoose models to interact with the MongoDB database.
- Implement CRUD operations (Create, Read, Update, Delete) for each model to perform database operations.

API Design and Development:

- Identify the necessary functionality and data required by the frontend.
- Design a set of RESTful APIs using a framework like Express.js or Django REST Framework.
- Define API endpoints for user management, product catalog, shopping cart, order management, payment gateway integration, shipping integration, etc.
- Implement the API routes, controllers, and data models to handle the corresponding operations.
- Ensure that the APIs follow best practices, are secure, and provide appropriate responses.

User Management and Authentication:

- Implement user registration and login functionality.
- Choose an authentication mechanism like session-based authentication or token-based authentication (e.g., JWT).
- Store and hash user credentials securely.
- Implement middleware to authenticate API requests and authorize access to protected routes.

Product Catalog and Inventory Management:

- Design the database schema to store product details, pricing, availability, and inventory levels.

- Create APIs to retrieve product information, update inventory quantities, and handle search and filtering.
- Implement validations to ensure data integrity and consistency.

Shopping Cart and Order Management:

- Design the database schema to store shopping cart details and order information.
- Create APIs to handle cart operations like adding items, modifying quantities, and placing orders.
- Implement logic to calculate totals, apply discounts, and manage the order lifecycle.

Payment Gateway Integration:

- Choose a suitable payment gateway provider (e.g., Stripe, COD).
- Integrate the payment gateway SDK or API to handle secure payment processing.
- Implement APIs or callback endpoints to initiate transactions, handle payment callbacks, and receive payment confirmation.

Shipping and Logistics Integration:

- Identify shipping and logistics providers that align with your application's requirements.
- Utilize the APIs provided by these providers to calculate shipping costs, generate shipping labels, and track shipments.
- Implement APIs or services to fetch rates, generate labels, and obtain tracking information.

Database Integration:

- Choose a suitable database technology (e.g., MySQL, PostgreSQL, MongoDB) based on your application's requirements.
- Design the database schema to efficiently store and retrieve flower and gift delivery data.
- Establish a connection to the database and handle data persistence and retrieval.

External Service Integration:

- Identify third-party services like email service providers, analytics services, or CRM systems that are required for your application.
- Utilize the APIs or SDKs provided by these services to exchange data and perform necessary operations.
- Implement the integration logic to send order confirmations, track user behavior, or manage customer relationships.

Security and Data Protection:

- Apply appropriate security measures like encryption techniques for secure data transmission and storage.
- Implement input validation and sanitization to prevent common security vulnerabilities.
- Implement access control to ensure authorized access to sensitive data.

Error Handling and Logging:

- Implement error handling mechanisms to handle exceptions and provide meaningful error messages to the frontend.
- Use logging frameworks to record application logs for monitoring and troubleshooting purposes.

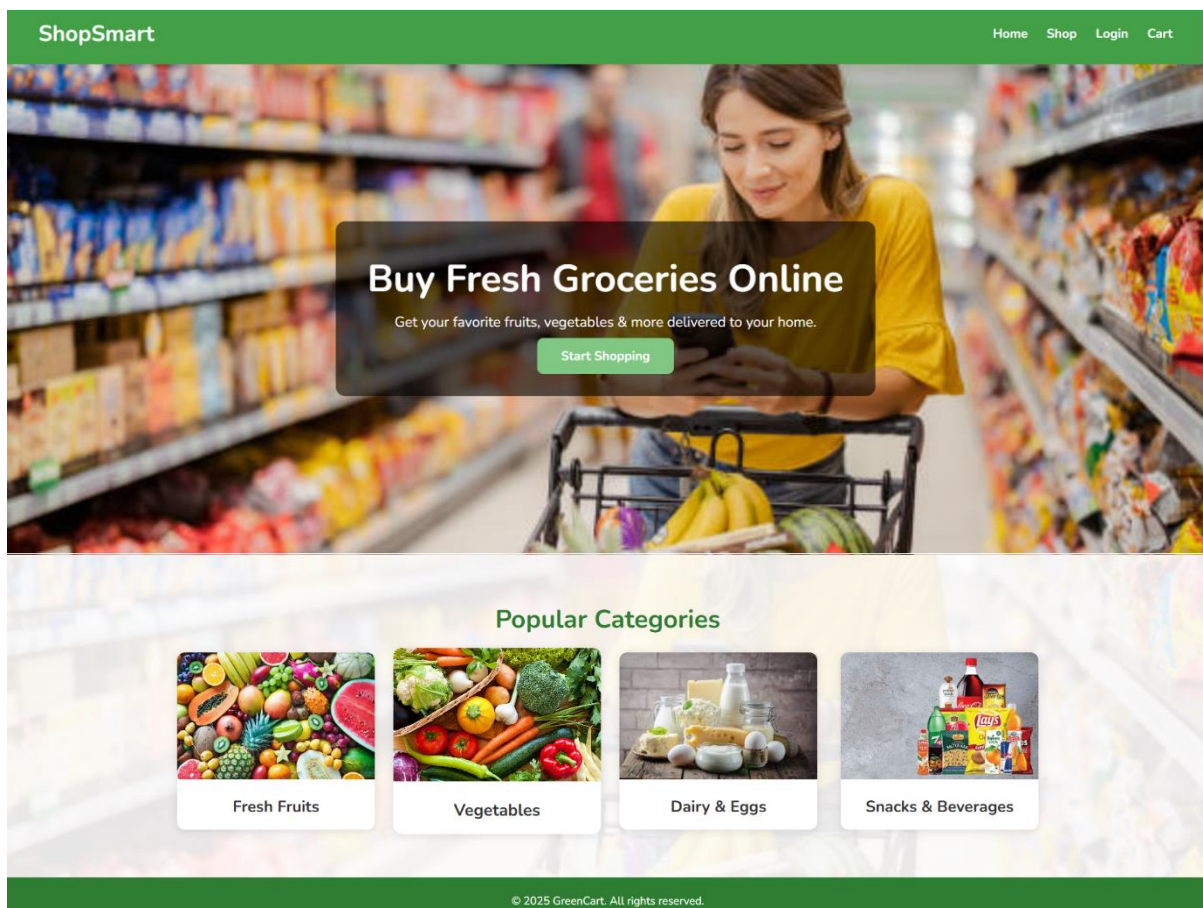
Connect database to backend:

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>ShopSmart - Digital Grocery Store</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f4f4f4; padding: 20px; }
    h1 { text-align: center; color: #2c3e50; }
    table { width: 100%; border-collapse: collapse; background: white; margin-top: 20px; }
    th, td { padding: 12px; text-align: center; border: 1px solid #ccc; }
    img { width: 80px; height: auto; border-radius: 8px; }
    button { margin: 2px; padding: 8px 12px; border: none; border-radius: 5px; cursor: pointer; }
    .add-btn { background-color: #28a745; color: white; }
    .buy-btn { background-color: #007bff; color: white; }
    .delete-btn { background-color: #dc3545; color: white; }
    #cart { margin-top: 30px; background: #fff; padding: 15px; border: 1px solid #ccc; }
    #cartItems li { margin: 5px 0; }
  </style>
</head>
```

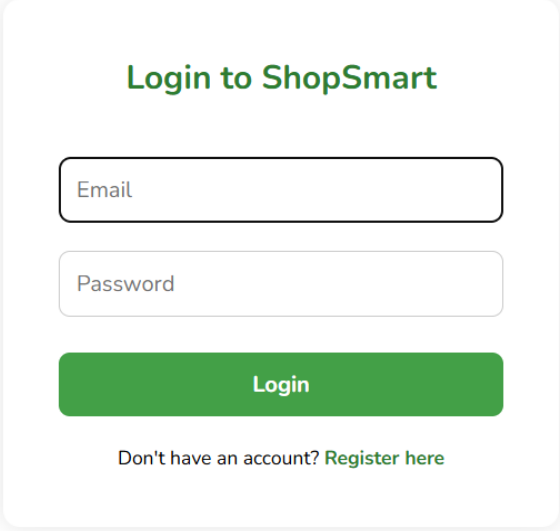
Integration:

Integration is the process of combining and connecting the frontend and backend components of the online flower shop web application to create a unified and fully functional system. It involves establishing communication channels, exchanging data, and ensuring seamless interaction between the frontend UI and backend APIs. The following activities are part of the integration process:

Landing page:-



Login page:

A login form titled "Login to ShopSmart" in green text. It features two input fields: "Email" and "Password", both with light gray borders and rounded corners. Below these fields is a solid green button with the text "Login" in white. At the bottom, there is a link that says "Don't have an account? Register here" in green text.

Login to ShopSmart

Email

Password

Login

Don't have an account? [Register here](#)

Items page:

Available Products



Apples

Price: \$1.20

Add to Cart

Buy Now



Milk

Price: \$3.49

Add to Cart

Buy Now



Bread

Price: \$4.99

Add to Cart

Buy Now



Eggs

Price: \$2.50

Add to Cart

Buy Now



Rice

Price: \$3.99

Add to Cart

Buy Now



Potatoes

Price: \$3.99

Add to Cart

Buy Now



Onions

Price: \$3.99

Add to Cart

Buy Now



Tomatoes

Price: \$3.99

Add to Cart

Buy Now



Chocolates

Price: \$3.99

Add to Cart

Buy Now



Dry Fruits

Price: \$3.99

Add to Cart

Buy Now

6 29°C
Mostly sunny



Q Search



My Cart:



Your Cart

- Apples - \$1.20 [Delete](#)
- Bread - \$4.99 [Delete](#)
- Potatoes - \$3.99 [Delete](#)
- Tomatoes - \$3.99 [Delete](#)
- Chocolates - \$3.99 [Delete](#)
- Sweets - \$3.99 [Delete](#)
- Detergents - \$3.99 [Delete](#)

My Orders page:

ShopSmart

HomeItemsCartMy OrdersLogin

My Orders

Order #ORD1023

Date: June 21, 2025

Items: Fresh Apples, Milk (2L), Wheat Bread

Total: ₹220

Delivered

Order #ORD1024

Date: June 24, 2025

Items: Eggs (6), Bananas (1kg)

Total: ₹120

Pending

Order #ORD1025

Date: June 25, 2025

Items: Tomatoes, Onions, Dairy Pack

Total: ₹150

Delivered

My History page:

Order History

Order #ORD1001

Date: May 10, 2025

Items: Rice, Onions, Potatoes

Total: ₹290

Delivered

Order #ORD1002

Date: May 22, 2025

Items: Milk, Cheese, Bread

Total: ₹180

Returned

Order #ORD1003

Date: June 2, 2025

Items: Eggs, Butter, Juice

Total: ₹220

Cancelled

Place Order page:

Place Your Order

Full Name

John Doe

Delivery Address

123, Street Name, City, ZIP

Phone Number

9876543210

Payment Method

Select Payment Method

Order Summary:

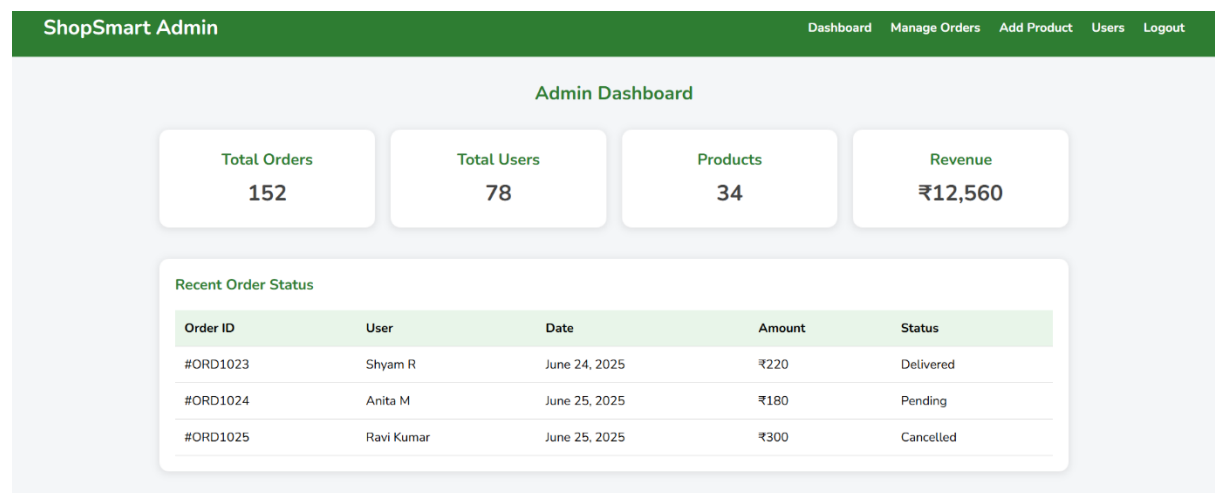
Items Total: ₹220

Delivery Charges: ₹30

Grand Total: ₹250

Confirm & Place Order

Admin Dashboard Page:



User Page:

ShopSmart Admin

DashboardOrdersAdd ProductUsersLogout

Registered Users

#ID	Full Name	Email	Joined On	Role
1	Ravi Kumar	ravi@example.com	2025-06-01	Customer
2	Anita Mishra	anita@example.com	2025-06-02	Admin
3	Shyam R	shyamr@example.com	2025-06-04	Customer

Add New Product:

ShopSmart Admin

DashboardOrdersAdd ProductUsersLogout

Add New Product

Product Name

Potatoes

Price (₹)

60

Image URL

e.g., https://example.com/image.jpg

Category

Vegetables

Description

Short description about the product

Add Product

Admin Orders Page:-

ShopSmart Admin

DashboardOrdersAdd ProductUsersLogout

Manage All Orders

Order ID	Customer	Items	Total	Date	Status	Action
#ORD1023	Ravi Kumar	Milk, Bread, Eggs	₹180	2025-06-24	Delivered	View
#ORD1024	Anita M	Onions, Apples	₹120	2025-06-25	Pending	View
#ORD1025	Shyam R	Bananas, Tomatoes	₹90	2025-06-25	Cancelled	View

The demo of the app is available:-

<https://drive.google.com/file/d/19T44fwnf4X0uTzNZGXFQp02w9Js52OJn/view?usp=drivesdk>

HAPPY SHOPPING!!

